

RASPUNSURI DE LA BIA AVRAM

The code below deletes the minimum element in a list.....how did the initial list L look like?

The code below deletes the minimum element in a list. We executed it on list L, which we know to look as `L=[6, . . . . .]`. Given that, according to the specific `write/writeln` statements, the execution displays what you see below, how did the initial list L look like?

```
del_min([H|T],[H|R],M):-
    del_min(T,R,M),
    M=<H,!,
    write(R), write(' '),writeln(M).
del_min([H|T],T,H) :-
    write(T), write(' '),writeln(H).

?-L=[6,...], del_min(L,New_List,Min).
```

```
[]4
[4]3
[3,4]1
[3,4]1
[3,3,4]1
[2,3,3,4]1
[1,2,3,3,4]1
```

R mai sus: aici e [6,1,2,3,1,3,4]

The code below deletes the minimum element in a list. We executed it on list L, which we know to look as `L=[7, . . . . .]`. Given that, according to the specific `write/writeln` statements, the execution displays what you see below, how did the initial list L look like?

```
del_min([H|T],[H|R],M):-
    del_min(T,R,M),
    M=<H,!,
    write(R), write(' '),writeln(M).
del_min([H|T],T,H) :-
    write(T), write(' '),writeln(H).

?-L=[7,...], del_min(L,New_List,Min).
```

```
[]8
[8]5
[5,8]1
[5,8]1
[3,5,8]1
[1,3,5,8]1
[2,1,3,5,8]1
```

Answer: L = [7, 2,1,3,5,8]

Raspunsul pt ex de mai sus e: L=[7,2,1,3,1,5,8] am testat in swish

A cut would freeze(prevent from backtracking) some nodes in the tree. It is NOT freezing

A cut would freeze (prevent from backtracking) some nodes in the tree. However, it is NOT freezing:

Select one:

- ☒ a. Its parent node after the cut is executed
- ☐ b. A subtree of its sibling node to its left before the cut is executed
- ☐ c. The subtree of its leftmost sibling node after the cut is executed
- ☐ d. Its leftmost sibling node after the cut is executed

[Clear my choice](#)

B E CORECT MAI SUS

To make the predicate `select_sort/3` stable, we need to

To make the predicate `select_sort/3` stable, we need to:

```
select_sort(In,Acc,Rez):-
    max_delete(In,In_without_Max,Max),
    select_sort(In_without_Max,[Max|Acc],Rez).
select_sort([],L,L).
```

```
max_delete([H|T],[H|R],M):-
    max_delete(T,R,M),
    M>H,!.
```

```
max_delete([H|T],T,H).
```

Select one:

- ☐ a. nothing, it is stable as it is
- ☐ b. swap the order of `max_delete(In,In_without_Max,Max)` and `select_sort(In_without_Max,[Max|Acc],Rez)`
- ☐ c. change the inequality into a nonstrict one
- ☐ d. add a clause for the empty list in `max_delete` predicate

Raspuns mai sus: C

The predicate swap can be used for bubble sort. The F variable:

The predicate swap can be used for bubble sort. The F variable:

```
swap([A,B|T],[B|R],F):-  
    A>B,!,  
    F=y,  
    swap([A|T],R,F).  
swap([A,B|T],[A|R],F):-  
    swap([B|T],R,F).  
swap([A],[A],n):-!.  
swap([A],[A],y):-!.
```

Select one:

- ☐ a. must be instantiated in the call, either to y or n
- ☐ b. gets instantiated on one of the two stopping conditions
- ☒ c. it gets re-instantiated whenever the first clause is reached during the execution
- ☐ d. gets instantiated to y in the first clause, then maintains its value throughout the execution

The predicate below implements bubble sort (where append/3 has the regular, correct implementation). How many calls does the second append make to get the sorted array for the input [2,1,5,4,3]:

The predicate below implements bubble sort (where append/3 has the regular, correct implementation). How many calls does the second append make to get the sorted array for the input [2,1,5,4,3]:

```
bubble_sort(In,Out):-  
    append(X,[A,B|Y],In),  
    A>B,!,  
    append(X,[B,A|Y],Int),  
    bubble_sort(Int,Out).  
bubble_sort(In,In).
```

Select one:

- ☒ a. 4
- ☐ b. 2
- ☐ c. 3
- ☐ d. 5

ASTEA SE FAC IN SWISH(DACA NU ESTI GENIU)

The predicate below has to verify if the first argument is present in the second argument

The predicate below has to verify if the first argument (an element) is present in the second argument. The following is true about the correctness of the predicate:

```
member(H, [H|_]) .  
member(X, [_|T]) :-  
    member(X, T) .  
member(_, []) :- !, fail.
```

Select one:

- ☐ a.
it is incorrect, because it fails when reaching the empty list.
- ☐ b. it is correct and all clauses are necessary.
- ☐ c. it is incorrect, because it always fails.
- ☒ d. it is correct, but the last clause is not necessary.

The predicate below intending to search for a given element in an incomplete Binary Tree. We need to do the following(pdf)

The predicate below is intending to search for a given element in an incomplete Binary Search Tree (ended in variable). Do do so, we need to do the following:

```
search_iBST(K, t(_, K, _)) :- ! .  
search_iBST(SK, t(L, K, _)) :-  
    SK < K, !,  
    search_iBST(SK, L) .  
search_iBST(SK, t(_, _, R)) :-  
    search_iBST(SK, R) .
```

Select one:

- ☐ a.
add the following clause `search_iBST(_, V) :- var(V), !, mandatory as the first clause.`
- ☐ b. add the following clause `search_iBST(_, V) :- var(V), !, fail.` anywhere in the code (first, second, third or fourth clause)
- ☒ c. add the following clause `search_iBST(_, V) :- var(V), !, fail.` mandatory as the first clause.
- ☐ d. add some clause for `nil`.

The predicate below concatenates two lists. To have a nondeterministic call, we need to:

???????????????????????????????? nu inteleg de ceee

The predicate below concatenates two lists. To have a nondeterministic call, we need to:

```
append([H|T], L, [H|R]) :-
```

```
    append(T, L, R).
```

```
append([], L, L).
```

Select one:

- ☐ a. change the order of the clauses
- ☒ b. make the initial call with the first 2 arguments free variables
- ☐ c. add a clause to decompose the second argument
- ☐ d. make the initial call with all 3 arguments free variables

The predicate below deletes one element from a list. It deletes deterministically because

Determinist- iti returneaza acelasi rezultat daca dai din nou

Nondeterminist- la al doilea apel returneaza alt rezultat

Question 1

Not yet answered

Marked out of 1.00

Flag question

The predicate below deletes one element from a list. It deletes deterministically because:

```
delete(H, [H|T], T) :- !.
```

```
delete(X, [H|T], [H|R]) :-
```

```
    delete(X, T, R).
```

Select one:

- ☐ a. it has just one recursive call
- ☐ b. it has no fact on the empty list
- ☒ c. it has a cut in the non-recursive clause
- ☐ d. it has no cut in the recursive clause

Which of the answers from below is correct and the most specific?

The list `[]` is ...

Select one:

- ☐ a. not an incomplete list as it contains only one single variable element.
- ☐ b. an incomplete list.
- ☐ c. not an incomplete list as it is missing the `|` pattern.
- ☐ d. not an incomplete list as there are no other elements in the list other than the final variable.

Raspuns b la intrebarea de mai sus

RESEARCH-Adi-GRUP

Cut , backtracking

Nodes frozen prevented from backtracking by cut

Question 1

Not yet answered

Marked out of 1.00

Flag question

Which categories of nodes are frozen (prevented from backtracking) by a cut:

Select one:

- ☒ a. Any node to its left before the execution of the cut
- ☐ b. Any sibling node to its right after the execution of the cut
- ☐ c. The parent node if the cut is the last subgoal
- ☐ d. Its parent node after the execution of the cut

A cut would freeze (prevent from backtracking) some nodes in the tree. However, it is NOT freezing:

Select one:

- ☐ a. Its parent node after the cut is executed
- ☐ b. The subtree of its leftmost sibling node after the cut is executed
- ☒ c. A sibling node to its left before the cut is executed
- ☐ d. Its leftmost sibling node after the cut is executed

A cut NEVER affects:

Select one:

- ☐ a. Its parent node
- ☐ b. A sibling node to its left
- ☒ c. Subtrees of subgoals to its right
- ☐ d. The subtree of a sibling node to its left

[Clear my choice](#)

Structure, unification

What is wrong with the following unification: $?- n(c(a,b), v, m(1,[2,b])) = n(c(a,b), v, m(1,[2|[3]]))$. ?

Select one:

- ☐ a. It fails, because $m(1,[2,b])$ cannot unify with $m(1,[2|[3]])$
- ☒ b. It fails, because the two structures do not have the same name
- ☐ c. The name of the structure on the right is not constant
- ☐ d. It fails, because v cannot unify with V

Tail,Head

When matching

When matching $X|Y$ with $[1,2,b]$ the result is:

Select one:

- ☒ a. Success, with $X=1$, $Y=[2,b]$
- ☐ b. Success, with $X=[1,2]$, $Y=b$
- ☐ c. Failure, as Y cannot become heterogeneous
- ☐ d. Failure, as the first list has 3 elements while the second 2 elements

The following matching: $[1,2,3]$ with $[A,B|C]$ would provide the following instantiations:

Select one:

- ☒ a. $A=1$, $B=2$, $C=[3]$
- ☐ b. The matching fails
- ☐ c. $A=1$, $B=2$, $C=3$
- ☐ d. No instantiation is provided

[Clear my choice](#)

The list $[A|_]$ is...

Which of the answers from below is correct and the most specific?

The list $[A|_]$ is ...

Select one:

- ☐ a. an incomplete list as A is a free variable
- ☐ b. a list with just one element (A) with an unknown value
- ☒ c. an incomplete list as its variable termination $_$ occurs after the pattern $|$.
- ☐ d. an incomplete list as it contains $_$

matching

The following matching: $f(A, g(B), 4)$ with $f(a, g, 4)$ would fail because:

Select one:

- ☐ a. 4 is the 3rd argument
- ☐ b. The matching actually succeeds
- ☒ c. Because of the second argument ($g(B)$ is an expression while g is a constant)
- ☐ d. A should be lower case

[Clear my choice](#)

the list $[A|_]$ may consist of incompleta-nu stim cate elemente vom mai avea

Which of the answers from below is correct and the most specific?

The list $[A|_]$ may consist of ...

Select one:

- ☒ a. at least 1 element
- ☐ b. exactly 1 element
- ☐ c. at least 2 elements
- ☐ d. exactly 2 elements

matching

The following matching: [A,b|C] with [1,2,3,4] fails because:

Select one:

- ☐ a. The first list has 3 elements while the second one 4 elements
- ☒ b.
Both b and 2 are constants (and different)
- ☐ c. The matching succeeds with instantiation: A=1, C=[3,4]
- ☐ d.
The first list has letters while the second one numbers

[Clear my choice](#)

Recursive clause, predicate, backward recursion, forward

The recursive clause below of some predicate is designed with backward recursion because:

```
p([H|T], Some_Result) :-  
    p(T, Some_Part_Result),  
    combines(H, Some_Part_Result, Some_Result).
```

Select one:

- ☐ a. No other operation is performed before the recursive call
- ☒ b.
It processes the current element after ending the recursion
- ☐ c. It only has 2 arguments, hence, cannot be forward
- ☐ d. It is NOT backward recursion as combines has 3 arguments

Recursive clause, predicate, backward recursion, forward

Consider a predicate having a recursive clause, as below:

```
p([H|T], Some_Result):-
```

```
    p(T, Some_Part_Result),
```

```
    combines(H, Some_Part_Result, Some_Result).
```

On the stopping condition:

Select one:

- ☐ a. We obtain the final result
- ☐ b. The result has to be made available at the top level (call level)
- ☐ c. The value of the second argument doesn't matter
- ☒ d. The result has to be initialized

replace, one occurrence, delete

replace one occurrence of the first argument from the list given in the second argument with the third

The predicate below is assumed to replace **one** occurrence of the first argument, from the list given in the second argument, with the third argument, to generate the list in the fourth argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
rep(X, [X|T], Y, [Y|T]).
```

```
rep(X, [H|T], Y, [H|R]) :-
```

```
    rep(X, T, Y, R).
```

```
?- rep(b, [a,b,c,b,d], z, R).
```

```
R= [a,z,c,b,d] ? ;
```

```
R= [a,b,c,z,d] ? ;
```

```
R= [a,b,c,b,d] ? ;
```

```
no
```

Select one:

- ☐ a. Update the clause 1 as `rep(X,[X|T],Y,R):-!,rep(X,T,Y,R).`
- ☐ b. Remove the clause `rep(X,[X|T],Y,[Y|T]).`
- ☐ c. Update the clause 2 as `rep(X,[H|T],Y,R):-!,rep(X,T,Y,R).`
- ☐ d. Add another clause like: `rep(X,[X]_,_[Y]).`
- ☐ e. Update the clause 1 as `rep(X,[X|T],Y,[Y|T]):-!.`
- ☐ f. Swap the order of clauses 1 and 2.
- ☐ g. Update the clause 2 as `rep(X,[H|T],Y,[H|R]):-!,rep(X,T,Y,R).`
- ☒ h. Add the clause `rep(_,[],_[]).`

a predicate which inserts an element in order into an ordered list, as last occurrence

Say we want to write a predicate which inserts an element, in order, into an ordered list, as **last occurrence**. Complete the partial implementation so that you get a correct predicate:

```
insertOrd(X, [H|T], ) :- X < H, , insertOrd(, , ) .  
insertOrd(X, T, ) .
```

`insert_ord(X, [H|T], [H|R]) :- X > H, !, insert_ord(X, T, R).`

`insert_ord(X, T, [X|T]).` **Pentru ex de mai sus**

remove exactly one occurrence of the first argument, from the list given in the second arg

The predicate below is assumed to remove **exactly one** occurrence (in turn) of the first argument, from the list given in the second argument, to generate the list in the third argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
delete(X, [X|T], T) .
```

```
delete(X, [H|T], [H|R]) :-
```

```
    delete(X, T, R) .
```

```
delete(_, [], []) .
```

```
?- delete(b, [a,b,c,b,d], R) .
```

```
R= [a,c,b,d] ? ;
```

```
R= [a,b,c,d] ? ;
```

```
no
```

Select one:

- ☐ a. Remove the clause `delete(X,[X|T],T).`
- ☐ b. Swap the order of clauses 1 and 2.
- ☐ c. Add another clause like: `delete(X,[X],[]).`
- ☐ d. Swap the order of clauses 1 and 3.
- ☒ e. Remove the clause `delete(_,[],[]).`
- ☐ f. Update the clause 2 as `delete(X,[H|T], R):-!,delete(X,T,R).`
- ☐ g. Update the clause 1 as `delete(X,[X|T],T):-!`.
- ☐ h. Update the clause 2 as `delete(X,[H|T],[H|R]):-!,delete(X,T,R).`

predicate to delete the last occurrence of the max value from a list

`max_delete([H|T], Max, [H|DT]) :- max_delete(T, Max, DT), H =< Max, !.`

`max_delete([H|T], H, T).`

Question 5
Not yet
answered
Marked out of
1.00
Flag question

The predicate below should be a predicate to delete the **last** occurrence of the max value from a list. Fill in the blanks for a correct predicate:

```
max_delete([H|T], , ) :-  
    max_delete(T, Max, DT),  
    H  Max,  
    .  
max_delete([H|T], , ).
```

The predicate replaces all occurrences replace_list in wordul cu cod

The predicate below replaces all occurrences of an element from a list with another element. It has an incorrect non-deterministic behavior, because:

```
replace(_,_, [], []).  
replace(H,A, [H|T], [A|R]) :-  
    replace(H,A,T,R).  
replace(X,A, [H|T], [H|R]) :-  
    replace(X,A,T,R).
```

Select one:

- ☐ a.
it has a clause for the empty list ✖
- ☒ b. it has no cut in the second clause
- ☐ c. it has no cut in the first clause
- ☐ d.
it has a recursive call in the second clause (it shouldn't)

partitions,pivot replaces occurrences

The predicate below replaces all occurrences of an element from a list with another element. It has an incorrect behavior, because:

```
replace(H,A, [H|T], [A|R]) :-  
    !,  
    replace(H,A,T,R).  
replace(X,A, [H|T], [H|R]) :-  
    replace(X,A,T,R).
```

Select one:

- ☐ a.
it has a recursive call in the second clause (it shouldn't)
- ☐ b.
it has a cut in the first clause
- ☐ c. it has no cut in the last clause
- ☒ d. it doesn't have a clause for the empty list

square,squared,reverse, receives

Say we want to write a predicate which receives as input a list of integers, and produces as output a list containing their squares. To obtain such a predicate, the recursive clause should look like:

Select one or more:

- ☐ a. `square([H|T], R):- square(T,R).`
- ☐ b. `square([H|T], Ri, R):- NH is H*H, square(T,[HN|Ri],R).`
- ☒ c. `square([H|T], [NH|R]):- NH is H*H, square(T,R).`
- ☐ d. `square([H|T], R):- NH is H*H, square(T,[HN|R]).`

Say we want to write a predicate which receives as input a list of integers, and produces as output a list containing their squared roots, in **reverse order**. To obtain such a predicate (correct and complete), the recursive clause should look like:

Select one or more:

- ☐ a. `square([H|T], R):- square(T,R).`
- ☐ b. `square([H|T], R):- NH is sqrt(H), square(T,[HN|R]).`
- ☐ c. `square([H|T], Ri, R):- NH is sqrt(H), square(T,[HN|Ri],R).`
- ☒ d. `square([H|T], [NH|R]):- NH is sqrt(H), square(T,R).`

difference of two sets

Complete the predicate below such that it performs the difference of two sets (represented as lists, member is considered known and implemented):

`d([H|T],L,R)` :- `member(H,L)` , `!` `d(T,L,R).`

`d([H|T],L,[H|R])` :- `d(T,L,R).`

`d([],L,[])` .

one pass,swaps adjacent numbers,

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). Use H whenever you need a NEW variable for an element from the list:

`one_pass([H1,H2|T], [H1,H2|R])` :- `H1 < H2` , `one_pass(T, R).`

`one_pass([H1,H2|T], [H2, H1|R])` :- `one_pass([H1|T], R).`

`one_pass([H1|T], [H1|R])` .

`one_pass([], [])` .

?- `one_pass([3,1,4,5,2], R).`

`R=[1,3,4,2,5];`

no.

```
one_pass([H1,H2|T], [H1,H2|R]):- H1<H2, !, one_pass(T, R).
one_pass([H1,H2|T], [H2|R]):- one_pass([H1|T], R).
one_pass([H], [H]).
one_pass([], []).
```

bst check-raspuns b ,minimum key, sum all leaf nodes, maximum,replace key

The following predicate is assumed to check if the tree is BST (Binary Search Tree):

```
is_BST(nil) .  
  
is_BST(t(t(LL,KL,LR),K,t(RL,KR,RR))):-  
    KL<K<KR,!,  
    is_BST(t(LL,KL,LR)),  
    is_BST(t(RL,KR,RR)).
```

Choose the most specific (and correct) answer from below:

Select one:

- ☐ a. the predicate is incorrect as there is no failure clause
- ☐ b. the predicate is incorrect, as the inequality check should consider other keys
- ☐ c. the predicate is correct
- ☐ d. the predicate is incorrect, as the inequality check should be reversed

sumLeaf, the sum of all leaf nodes in a BST

I want to write a predicate which finds the sum of all leaf nodes in a BST. Complete the partial implementation so that you get a correct predicate:

```
sumLeaf(nil,0).  
sumLeaf(t(K,nil,nil),K).  
sumLeaf(t(K,L,R),Sum):-  
    sumLeaf(L,SL),  
    sumLeaf(R,SR),  
    Sum is SL + SR.
```

Maximum sum on a branch of a BST

I want to write a predicate which finds the maximum sum on a branch of a BST. Complete the partial implementation so that you get a correct predicate:

```
maxSum(nil,0).  
maxSum(t(K,L,R),Sum):-  
    maxSum(L,SL),  
    maxSum(R,SR),  
    Sum is max(SL,SR).
```

Check if a tree is BST Binary Search Tree

A predicate to check if a tree is BST (Binary Search Tree) written like:

```
is_BST(nil) .  
is_BST(t(L,K,R)) :-  
    max(L,ML) ,  
    min(R,MR) ,  
    ML<K<MR.
```

where `max/2` returns the max key, while `min/2` returns the min key (from a tree), is:

Select one:

- ☒ a. incorrect as it only checks the condition in the root
- ☐ b. incorrect as K should be compared to the keys in the roots of L and R
- ☐ c. incorrect as it has no output argument
- ☐ d. correct as it checks the conditions for being BST

[Clear my choice](#)

Minimum key in a BST Binary Search Tree

I want to write a predicate which computes the minimum key in a BST. Complete the partial implementation so that you get a correct predicate:

```
min(t(K,_,_), _) :-  
< min(t(K,L,R), M):-min( , ).
```

Insert a key in an incomplete BST Binary Search Tree

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```
ins(t(_,Key,_),Key) :- !.  
ins(t(L,Key1,_),Key) :-  
    Key1>Key, !,  
    ins(L,Key) .  
ins(t(_,Key1,R),Key) :-  
    ins(R,Key) .
```

Select one:

- ☒ a. The predicate is correct.
- ☐ b.
The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached).
- ☐ c. The predicate is incorrect, as it's missing the output argument
- ☐ d.
The predicate is incorrect as the second clause must be the first one.

[Clear my choice](#)

collect in order, incomplete binary search tree, collect_2_children

The following code is assumed to collect, in order, the internal nodes with 2 children of an incomplete binary search tree. Choose the most specific answer from below:

```
collect_2_children(t(L,_,R),N):-var(L),!,
    collect_2_children(R,N).
collect_2_children(t(L,_,R),N):-var(R),!,
    collect_2_children(L,N).
collect_2_children(t(L,Key,R),NN):-
    collect_2_children(R,NR),
    collect_2_children(L,NL),
    append(NL,[Key|NR],NN).
collect_2_children(t(L,_,R),[]):-
    var(L),var(R),!.
```

Select one:

- ☐ a. The predicate is incorrect, as is missing a clause on `var(T)`.
- ☐ b. The predicate is correct.
- ☐ c. The predicate is incorrect as the clause right subtree is processed before the left one.
- ☒ d. The predicate is incorrect as the last clause should be the first one.

Insert a key in an incomplete BST , ins

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below.

```
ins(t(L,Key1,_),Key):-
    Key1>Key,!,
    ins(L,Key).
ins(t(_,Key1,R),Key):-
    ins(R,Key).
ins(t(_,Key,_),Key):-!.
```

Select one:

- ☐ a. The predicate is incorrect, as it's missing the output argument
- ☐ b. The predicate is incorrect as the second clause is missing the comparison `Key1<Key`.
- ☐ c. The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached).
- ☒ d. The predicate is incorrect as the last clause must be the first one.

collect_2_children, binary search tree, internal nodes

The following code is assumed to collect, in order, the internal nodes with 2 children of a binary search tree. Choose the most specific answer from below:

```
collect_2_children(t(L,_,R),NN):-
    collect_2_children(L,NL),
    collect_2_children(R,NR),
    append(NR,NL,NN).
collect_2_children(t(nil,_,R),N):-
    collect_2_children(R,N).
collect_2_children(t(L,_,nil),N):-
    collect_2_children(L,N).
collect_2_children(t(nil,_,nil),[]).
```

Select one:

- ☐ a. The predicate is correct.
- ☐ b. The predicate is incorrect, as is missing a clause on nil.
- ☐ c. The predicate is incorrect as the clause with 2 recursive calls is not adding the node to the output list
- ☒ d. The predicate is incorrect as the last clause should be the first one.

maximum key in a BST, max

I want to write a predicate which computes the maximum key in a BST. Complete the partial implementation so that you get a correct predicate:

```
max(t(K,_,_),K).
max(t(K,L,R),M):-max(L,M).
```

collect, 2 children, binary search tree, collect_2_children

The following code is assumed to collect, in order, the internal nodes with 2 children of a binary search tree. Choose the most specific answer from below:

```
collect_2_children(t(L,_,R),NN):-
    collect_2_children(L,NL),
    collect_2_children(R,NR),
    append(NR,NL,NN).
collect_2_children(t(nil,_,R),N):-
    collect_2_children(R,N).
collect_2_children(t(L,_,nil),N):-
    collect_2_children(L,N).
collect_2_children(t(nil,_,nil),[]).
```

Select one:

- ☐ a. The predicate is correct.
- ☐ b. The predicate is incorrect, as is missing a clause on nil.
- ☒ c. The predicate is incorrect as the clause with 2 recursive calls is not adding the node to the output list
- ☐ d. The predicate is incorrect as the last clause should be the first one.

collect_2_children , incomplete binary search tree

The following code is assumed to collect, in order, the internal nodes with 2 children of an incomplete binary search tree. Choose the most specific answer from below:

```
collect_2_children(t(L,_,R),N):-var(L),!,
    collect_2_children(R,N).
collect_2_children(t(L,_,R),N):-var(R),!,
    collect_2_children(L,N).
collect_2_children(t(L,Key,R),NN):-
    collect_2_children(R,NR),
    collect_2_children(L,NL),
    append(NL,[Key|NR],NN).
collect_2_children(t(L,_,R),[]) :-
    !,var(L),var(R),!.
```

Select one:

- ☒ a. The predicate is incorrect as the last clause should be the first one.
- ☐ b. The predicate is correct.
- ☐ c. The predicate is incorrect as the clause right subtree is processed before the left one.
- ☐ d. The predicate is incorrect, as is missing a clause on var (T) .

replace a given key in a BST with another key, replace

I want to write a predicate which replaces a given key in a BST with another key, specified as argument. The predicate should leave the tree unchanged if the searched key is not in the tree. Complete the partial implementation so that you get a correct predicate:

```
replace(_,nil,nil).
replace(K,NK,t(K,L,R),t(NK,L,R)):-!.
replace(K,NK,t(Key,L,R),t(Key,NL,R)):-
    K<Key,
    !,
    replace(K,NK,L,NL).
replace(K,NK,t(Key,L,R),t(Key,L,NR)):-
    replace(K,NK,R,NR).
```

collect_internal, internal nodes, incomplete BST, difference lists

Question 6

Not yet answered

Marked out of 1.00

Flag question

The following code is assumed to collect, in order, the internal nodes (non leaves) of an incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```
collect_internal (L,_,R),B,R):-
    var(L),var(R),!.
collect_internal (L,Key,R),B,R):-
    var(R),!,
    collect_internal (L,B,[Key|R]),
collect_internal (L,Key,R),B,R):-
    var(L),!,
    collect_internal (R,[Key|B],E),
collect_internal (L,Key,R),BRes,ERes):-
    collect_internal (R,Int,ERes),
    collect_internal (L,BRes,[Key|Int]).
```

Select one:

- ☒ a. The predicate is incorrect because the pattern of adding the `Key` in the third clause (on the right list `R`) `[Key|B]` must occur in the head of the rule.
- ☐ b. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- ☐ c. The predicate is incorrect because the pattern of adding the `Key` in the second clause (on the left list `L`) `[Key|B]` must occur in the head of the rule.
- ☐ d. The predicate is incorrect, as is missing a first clause on `var(T)`.

[Clear my choice](#)

Calculate, count_2_children, binary tree, internal nodes

b

Question 2

Not yet answered

Marked out of 1.00

Flag question

The following code is assumed to calculate the internal nodes with 2 children of a binary tree. Choose the most specific answer from below:

```
count_2_children (L,_,R),NN):-
    count_2_children (L,NL),
    count_2_children (R,NR),
    NN is NL+NR.
count_2_children (nil,_,R),N):-
    count_2_children (R,N).
count_2_children (L,_,nil),N):-
    count_2_children (L,N).
count_2_children (nil,_,nil),0).
```

Select one:

- ☐ a. The predicate is incorrect as the clause with 2 recursive calls is not counting the node
- ☐ b. The predicate is incorrect as the last clause should be the first one.
- ☐ c. The predicate is incorrect, as is missing a clause on `nil`.
- ☐ d. The predicate is correct.

collect, non-empty incomplete BST, difference lists, collect_leaves

d

Question 3

Not yet answered

Marked out of 1.00

Flag question

The following code is assumed to collect, in order, the leaves of a **non-empty** incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```

collect_leaves (t (L, Key, R), B, [Key|R]) :-
    var (L), var (R), !.
collect_leaves (t (Lr, R), B, E) :-
    var (R), !,
    collect_leaves (Lr, B, E).
collect_leaves (t (Lr, R), B, E) :-
    var (L), !,
    collect_leaves (R, B, E).
collect_leaves (t (L, Key, R), BRes, ERes) :-
    collect_leaves (R, Lnt, ERes),
    collect_leaves (L, BRes, Lnt).
        
```

Select one:

- ☐ a. The predicate is incorrect, as is missing a first clause on var (T) .
- ☐ b. The predicate is incorrect as clauses 2 and 3 should occur in the opposite order.
- ☐ c. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- ☐ d. The predicate is incorrect because the pattern of adding the leaf key is incorrect.

insert a key in an incomplete BST, ins

b

Question 6

Not yet answered

Marked out of 1.00

Flag question

The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below:

```

ins (t (_, Key, _), Key) :- !.
ins (t (L, Key1, R), Key) :-
    var (R),
    Key1 > Key, !,
    ins (L, Key).
ins (t (L, Key1, R), Key) :-
    var (L),
    !,
    ins (R, Key).
        
```

Select one:

- ☐ a. The predicate is incorrect, as it's missing the output argument
- ☐ b. The predicate is incorrect as it does not handle the general case (with both L and R nonvar).
- ☐ c. The predicate is incorrect as it does not treat the case when the incomplete tree (var (T)) is reached.
- ☐ d. The predicate is incorrect as the third clause is missing the comparison Key1 > Key and the first clause must be the last one.

member, first occurrence, incomplete list

The following predicate should search for the first occurrence of an element in an *incomplete list*. It:

```
member(X, [X|_]) :- !.  
member(X, L) :- var(L), !, fail.  
member(X, [_|T]) :- member(X, T).
```

Select one:

- ☐ a. is not correct, because it fails when reaching the variable at the end of the list
- ☐ b. is correct, because the condition for reaching the variable at the end of the list is explicit, and contains definitive failure
- ☒ c. is not correct, because the clause for reaching the variable at the end of the list is not the first
- ☐ d. is not correct, because it never finds the searched element

[Clear my choice](#)

search operation, incomplete list, predicate, regular list, member

In a search operation in an incomplete list we may use the search predicate in a regular list (`member`), if we do the following change:

Select one:

- ☐ a. add a last clause to check when reached the final variable: `var(V), fail, !.`
- ☐ b. add a first clause to check when reached the final variable: `var(V), fail, !.`
- ☒ c. add a first clause to check when reached the final variable: `var(V), !, fail.`
- ☐ d. add a last clause to check when reached the final variable: `var(V), !, fail.`

member, complete lists, incomplete lists

The following implementation of the member predicate (for complete lists) does not behave correctly on *incomplete lists*, because:

```
member(X, [X|_]) :- !.
```

```
member(X, [_|T]) :- member(X, T).
```

Select one:

- ☐ a. it is deterministic
- ☐ b. it never finds the searched element
- ☒ c. it should fail explicitly when the search reaches the variable at the end of the list
- ☐ d. it does not have a clause for when the input list becomes []

member, deep list, correctness

The predicate below has to verify if the first argument (an element) is present in the second argument (a deep list). The following is true about the correctness of the predicate:

```
member(H, [H|_]).
```

```
member(X, [H|_]) :-
```

```
    member(X, H).
```

```
member(X, [_|T]) :-
```

```
    member(X, T).
```

```
member(_, []) :- !, fail.
```

Select one:

- ☒ a. it is correct, but the last clause is not necessary. ✓
- ☐ b. it is correct and all clauses are necessary.
- ☐ c. it is incorrect, because it searches in the head of the list also
- ☐ d. it is incorrect, because it fails when reaching the empty list.

intending, member_IL, search, incomplete list

The predicate below is intending to search for a given element in an incomplete list (list terminated in a variable). Do do so, we need to do the following:

```
member_IL(H, [H|_]) .  
member_IL(X, [_|T]) :-  
    member_IL(X, T) .
```

Select one:

- ☐ a. add some clause for the empty list.
- ☐ b. add the following clause `member_IL(_,V) :-var(V),!,fail.` mandatory as the first clause.
- ☒ c. add the following clause `member_IL(_,V) :-var(V),!,fail.` mandatory as the first clause ✓
- ☐ d. add the following clause `member_IL(_,V) :-var(V),!,fail.` anywhere in the code (first, second or third clause)

deep list, member, correctness of a predicate

The predicate below has to verify if the first argument (an element) is present in the second argument (a deep list). The following is true about the correctness of the predicate:

```
member(H, [H|_]) .  
member(X, [H|_]) :-  
    member(X, H) .  
member(X, [_|T]) :-  
    member(X, T) .  
member(_, []) :-!,fail.
```

Select one:

- ☒ a. it is correct, but the last clause is not necessary. ✓
- ☐ b. it is correct and all clauses are necessary.
- ☐ c. it is incorrect, because it searches in the head of the list also
- ☐ d. it is incorrect, because it fails when reaching the empty list.

Member, deep list, incomplete deep list

The predicate below has to verify if the first argument (an element) is present in the second argument (a deep list). If the second argument is an incomplete deep list (ended in variable, at all levels):

```
member(H, [H|_]) .  
member(X, [H|_]) :-  
    member(X, H) .  
member(X, [_|T]) :-  
    member(X, T) .
```

Select one:

- ☒ a. verifies if the first argument is present in the second argument or else it adds the element at the end of the first incomplete list found in the structure
- ☐ b. it adds the element at the end of all the incomplete lists in the structure
- ☐ c. it fails.
- ☐ d. it does the same: verifies if the first argument is present in the second argument. ✖

member, verify if the first argument is present in the second, deep list, deterministic

The predicate below has to verify if the first argument (an element) is present in the second argument (a deep list). To make the predicate deterministic, one must:

```
member(H, [H|_]) .  
member(X, [H|_]) :-  
    member(X, H) .  
member(X, [_|T]) :-  
    member(X, T) .
```

Select one:

- ☐ a. add a ! on the first clause and remove the second clause
- ☐ b. add a ! on the first clause
- ☒ c. add a ! on the first clause and another ! at the end of the second clause
- ☐ d. add a ! at the end of the second clause

elements,list , [A,B|_], elements

Which of the answers from below is correct and the most specific?

The list [A,B|_] may consist of ...

Select one:

- ☐ a. exactly 2 elements
- ☐ b. exactly 3 elements
- ☒ c. at least 2 elements
- ☐ d. at least 3 elements

[Clear my choice](#)

a

incomplete list

The predicate below is intending to search for a given element in an incomplete list (list terminated in a variable). Do do so, we need to do the following:

```
member_IL(H, [H|_]) .  
member_IL(X, [_|T]) :-  
    member_IL(X, T) .
```

Select one:

- ☐ a. add some clause for the empty list.
- ☐ b.
add the following clause `member_IL(_,V) :-var(V), !, mandatory as the first clause.`
- ☒ c. add the following clause `member_IL(_,V) :-var(V), !, fail.` mandatory as the first clause ✓
- ☐ d. add the following clause `member_IL(_,V) :-var(V), !, fail.` anywhere in the code (first, second or third clause)

adds an element at the end of some incomplete list

Say we want to write a predicate which adds an element at the end of some *incomplete list*, given as argument. The predicate should:

Select one:

- ☐ a. traverse the given list, instantiate an additional result variable accordingly when reaching the end of the list, then copy all the input list elements in the result, as recursion returns, to maintain their order
- ☒ b. traverse the given list, to reach the variable at the end, then instantiate that variable accordingly
- ☐ c. traverse the given list, and add the elements of the input list, one by one, in an additional result argument, as recursion proceeds, making sure to add the element at the end of the result list (when recursion ends)
- ☐ d. directly instantiate the variable at the end of the given list, accordingly (no traversal needed)

search for the first occurrence of an element in an incomplete list

C

Question 4

Not yet answered

Marked out of 1.00

Flag question

The following predicate should search for the first occurrence of an element in an *incomplete list*. It:

```
member(X, L) :- var(L), fail.
```

```
member(X, [X|_]) :- !.
```

```
member(X, [_|T]) :- member(X, T).
```

Select one:

- ☒ a. is not correct, because it never finds the searched element
- ☐ b. is not correct, because it is deterministic
- ☐ c. is not correct, because backtracking is allowed when reaching the variable at the end of the list
- ☐ d. is correct, because the condition for reaching the end of the list is the first, and contains definitive failure

[Clear my choice](#)

sort, traverse bubble sort ins, decreasing, condition-controlled

The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort. Would it be a sound choice (that is, can be bubble sort implemented using it)?

```
traverse_once([A,B|T],[B|R]) :-  
    A>=B,!,  
    traverse_once([A|T],R).  
traverse_once([A|T],R) :-  
    traverse_once(T,R).  
traverse_once([],[]).
```



Select one:

- ☐ a. No! It (traverse_once) fails as it cannot reach the clause with the empty list due to patterns in the other clauses.
- ☐ b. No! It (traverse_once) fails as fact should come as first clause.
- ☐ c. Yes! If properly employed, that is, until no swap occurred in the latest traverse_once.
- ☐ d. No! A bubble sort using it might enter an infinite loop in case duplicates exist in the input.

append, bubbleSort, swapping

If the code below uses append with the fact as the first clause, for the input [3,6,2,4,1] what would be the second selection of A and B for swapping (setting B,A) during execution?

```
bubbleSort(In, Out) :-  
    append(X, [A, B|Y], In),  
    A > B, !,  
    append(X, [B, A|Y], Int),  
    bubbleSort(Int, Out).  
bubbleSort(In, In).
```

Select one:

- ☐ a. A=2, B=4
- ☐ b. A=6, B=4
- ☐ c. A=3, B=6
- ☒ d. A=3, B=2

decreasing sorting by selection strategy, max, delete, sort

If we want a **decreasing** sorting by selection strategy (below), where `max` and `delete` are correct predicates to find the max value, and delete a value from a list respectively, which condition must hold true (choose the most specific one):

```
sort(In, [M|Out]) :-  
    max(In, M),  
    delete(M, In, Int),  
    sort(Int, Out).  
sort([], []).
```

Select one:

- ☒ a. `delete` needs to delete the first occurrence of the value from the list
- ☐ b. `max` needs to choose the first occurrence of the largest value from the list
- ☐ c. `min` needs to use a nonstrict inequality
- ☐ d. `min` needs to be implemented with backward recursion

In a decreasing sorting by selection strategy (below), the `max_delete` predicate is a stable predicate which correctly deletes the maximal value from the list. If we print out, what would be the second printed list on input `In = [3, 2, 5, 1, 5]`?

```
sort(In, [M|Out]) :-  
    max_delete(M, In, Int),  
    sort(Int, Out).  
sort([], []).
```

Select one:

- ☒ a. [3, 2]
- ☒ b. [5, 5]
- ☐ c. [1]
- ☐ d. [5]

[Clear my choice](#)

Sorting by selection strategy, min, delete

If we want a sorting by selection strategy (below), where `min` and `delete` are correct predicates to find the min value, and delete a value from a list respectively, which condition must hold true (choose the most specific one):

```
sort(In, [M|Out]) :-  
    min(In, M),  
    delete(M, In, Int),  
    sort(Int, Out).  
  
sort([], []).
```

Select one:

- ☐ a. `min` needs to use a nonstrict inequality
- ☐ b. `min` needs to choose the first occurrence of the smallest value from the list
- ☐ c. `min` needs to be implemented with backward recursion
- ☒ d. `delete` needs to delete the first occurrence of the value from the list

decreasing sorting by selection strategy, max_delete, stable

In a decreasing sorting by selection strategy (below), the `max_delete` predicate is a stable predicate which correctly deletes the maximal value from the list. If we print `Out`, what would be the second printed list on input `In` `=[7,3,5,3,7]`?

```
sort(In, [M|Out]) :-  
    max_delete(M, In, Int),  
    sort(Int, Out).  
sort([], []).
```

Select one:

- ☐ a. [3,3]
- ☐ b. [7]
- ☒ c. [3]
- ☐ d. [7,7]

bubble sort ,append, how many calls,

The predicate below implements bubble sort (where `append/3` has the regular, correct implementation). How many calls does the second `append` make to get the sorted array for the input `[2,3,1,4]`:

```
bubble_sort(In, Out) :-  
    append(X, [A, B|Y], In),  
    A > B, !,  
    append(X, [B, A|Y], Int),  
    bubble_sort(Int, Out).  
bubble_sort(In, In).
```

Select one:

- ☐ a. 4
- ☐ b. 5
- ☒ c. 2
- ☐ d. 3

bubble sort, regular, append

The predicate below implements bubble sort (where `append/3` has the regular, correct implementation). How many calls does the second `append` make to get the sorted array for the input `[2,5,1,3,4]`:

```
bubble_sort(In, Out) :-  
    append(X, [A, B|Y], In),  
    A > B, !,  
    append(X, [B, A|Y], Int),  
    bubble_sort(Int, Out).  
  
bubble_sort(In, In).
```

Select one:

- ☐ a. 2
- ☒ b. 4
- ☐ c. 5
- ☐ d. 3

swap, bubble sort

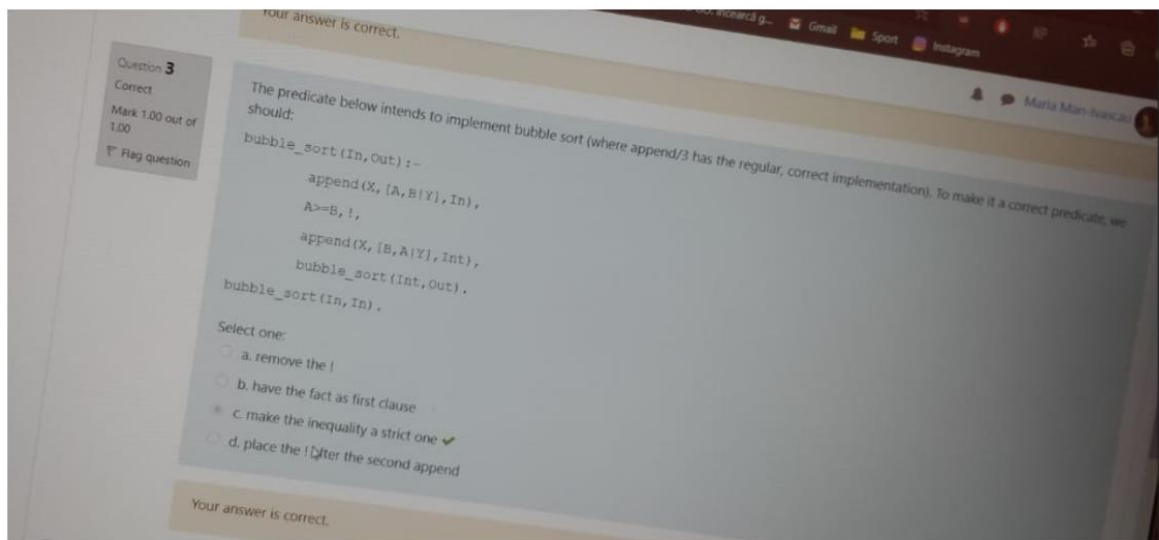
The predicate `swap` can be used for bubble sort. The `F` variable:

```
swap([A, B|T], [B|R], F) :-  
    A > B, !,  
    F = 0,  
    swap([A|T], R, F).  
  
swap([A, B|T], [A|R], F) :-  
    swap([B|T], R, F).  
  
swap([A], [A], 1) :- !.  
swap([A], [A], 0) :- !.
```

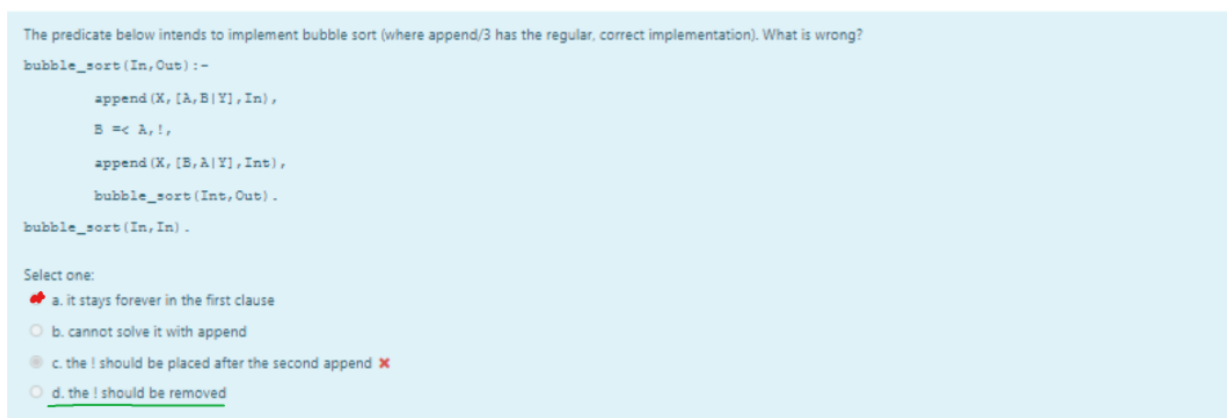
Select one:

- ☐ a.
must be instantiated in the call, either to 0 or 1
- ☒ b.
it gets re-instantiated whenever the first clause is reached during the execution ✖
- ☐ c. gets instantiated on one of the two stopping conditions
- ☒ d. gets instantiated to 0 in the first clause, then maintains its value throughout the execution

bubble sort, append,



bubble sort, regular, correct, what is wrong ?



assert, insct, dynamic

Which of the following assert sequences can generate the given clause order for the dynamic predicate insect/1:

```
insect(ant).  
insect(bee).  
insect(moth).  
insect(fly).  
insect(mosquito).
```

Select one:

- ☐ a. `asserta(insect(moth)) -> asserta(insect(bee)) -> assertz(insect(ant)) -> assertz(insect(fly)) -> assertz(insect(mosquito))`
- ☐ b. `assert(insect(mosquito)) -> asserta(insect(fly)) -> assert(insect(moth)) -> asserta(insect(bee)) -> asserta(insect(ant))`
- ☐ c. `assert(insect(moth)) -> asserta(insect(bee)) -> assert(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))`
- ☒ d. `asserta(insect(bee)) -> assert(insect(moth)) -> asserta(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))`

stable , instable, delete the max value from a list, blanks, max_delete

The predicate below should be a **stable** predicate to delete the max value from a list. Fill in the blanks for a correct predicate:

```
max_delete([X|T], M, [X|R]) :-  
    max_delete(T, M, R),  
    X < M,  
    !.  
max_delete([X|T], X, T).
```

ins_sort/2, stable

To make the predicate `ins_sort/3` stable, we need to:

```
ins(X, [], [X]).  
ins(X, [H|T], [X,H|T]) :-  
    H>X, !.  
ins(X, [H|T], [H|R]) :-  
    ins(X, T, R).
```

```
ins_sort([A|T], Int, Out) :-  
    ins(A, Int, NewInt),  
    ins_sort(T, NewInt, Out).  
ins_sort([], Out, Out).
```

Select one:

- ☒ a. nothing, it is stable as it is
- ☐ b. swap the order of `ins_sort(T, NewInt, Out)` and `ins(A, Int, NewInt)` ✗
- ☐ c.
change the strict inequality into a nonstrict one
- ☐ d. remove the clause for the empty list in `ins` predicate

ins_sort/2, stable, ins

To make the predicate `ins_sort/2` stable, we need to:

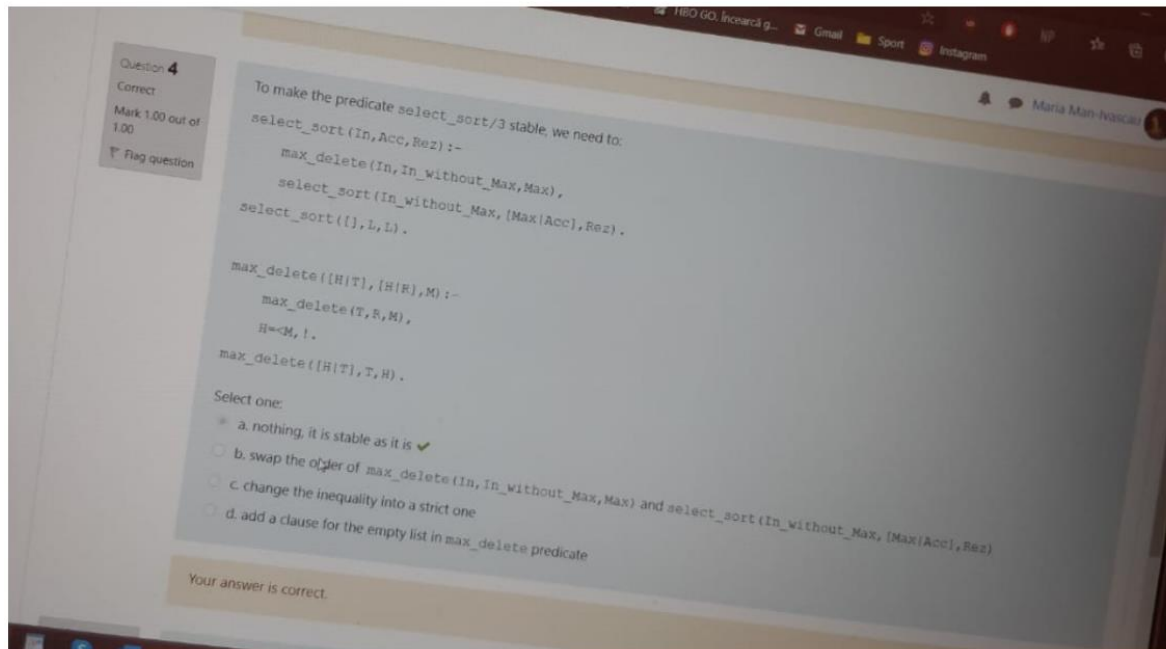
```
ins([A|T], B, [A|R]) :-  
    B>A, !,  
    ins(T, B, R).  
ins(L, B, [B|L]).
```

```
ins_sort([A|T], Out) :-  
    ins_sort(T, Int),  
    ins(Int, A, Out).  
ins_sort([], []).
```

Select one:

- ☐ a. change the strict inequality into a nonstrict one
- ☐ b. add a clause for the empty list in `ins/3` predicate
- ☐ c. swap the order of `ins_sort(T, Int)` and `ins(Int, A, Out)`
- ☒ d. nothing, it is stable as it is ✓

select_sort/3, stable, max_delete



ins_sort/2 stable

To make the predicate ins_sort/2 stable, we need to:

```
ins(X, [], [X]).
ins(X, [H|T], [X, H|T]) :-
    X < H, !.
ins(X, [H|T], [H|R]) :-
    ins(X, T, R).
```

```
ins_sort([A|T], Out) :-
    ins_sort(T, Int),
    ins(A, Int, Out).
ins_sort([], []).
```

Select one:

- ☐ a. remove the clause for the empty list in ins predicate
- ☒ b. change the strict inequality into a nonstrict one
- ☐ c. swap the order of ins_sort(T, Int) and ins(A, Int, Out)
- ☐ d. nothing, it is stable as it is ✗

checking, verify, graph, path, cycle, constructing, find cycles in a graph containing the node X

Complete the following predicate to make it find cycles in a graph containing the node X:

```
c(X, [FFX|P]) :- edge(X, Y), c(Y[X, Y], P).
```

```
c(X, Y, PP, FP) :-
```

```
    edge(X, Z),
```

```
    \+(member(Z, PP)),
```

```
    c(Z, Y, [Z|PP], FP).
```

```
c(X, X, PP, P) :- reverse(PP, P).
```

undirected graph, edge, checking, vertex, verify

d

Question 7

Not yet answered

Marked out of 1.00

Flag question

In an undirected graph represented by its edges (e.g. `is_edge(a,b)`), the predicate checking from below is assumed to check that each vertex in the graph is possessing a certain property which is soundly checked by the `verify(Vertex)` predicate. Assuming that `verify` is correctly implemented, choose the most specific answer from below.

```
checking:- is_edge(X,_),
           not(v(X)),
           assertz(v(X)),
           verify(X),!,
           fail.
checking.
```

Select one:

- ☐ a. The predicate is correct if the `!` is removed.
- ☐ b. The predicate is correct as it is.
- ☐ c. The predicate is correct if `assertz` is replaced by `asserta`.
- ☐ d. The predicate is correct if a negation (`not`) is added in front of (`verify`).

undirected graph represented by its edges, checking, possessing, verify(Vertex)

In an undirected graph represented by its edges (e.g. `is_edge(a,b)`), the predicate `checking` from below is assumed to check that each vertex in the graph is possessing a certain property which is soundly checked by the `verify(Vertex)` predicate. Assuming that `verify` is correctly implemented, choose the most specific answer from below:

```
checking:- is_edge(X,_),
           not(v(X)),
           assertz(v(X)),
           not(verify(X)),!,
           fail.
checking.
```

Select one:

- ☐ a. The predicate is correct if `assertz` is replaced by `asserta`.
- ☐ b. The predicate is correct if the negation (`not`) in front of (`verify`) is removed.
- ☐ c. The predicate is correct if the `!` is removed.
- ☒ d. The predicate is correct as it is.

finds a path in a graph, having the length smaller than M

Complete the predicate below such that it finds a path in a graph, having the length smaller than M:

`p(X,Y,M,P):- p(X,Y,M,[X],P).`

`p(X, , M, PP, PP):-`

`length(, L),`

`< M.`

`p(X, , M, PP, FP):-`

`edge(X,Z),`

`\+(member(, PP)),`

`p(, Y, M, , FP).`

Make it find cycles in a graph containing the node X

`c(X, [X|P]) :- edge(X,Y), c(Y,X,[Y],P).`

`c(X,Y,PP,FP) :- edge(X,Z), member(Z,PP), c(Z,Y,[Z|PP],FP).`

`c(X,X,PP,P) :- reverse(PP,P).`

Question 4

Not yet answered

Marked out of 1.00

Flag question

Complete the following predicate to make it find cycles in a graph containing the node X

`c(X, [ ] | P) :- edge(X,Y), c(Y,X,[Y],P).`

`c(X,Y,PP,FP) :-`

`edge( , Z),`

`\+ (member( , FP)),`

`c( , , [Z|PP], FP).`

`c(X, , PP,P) :- reverse(PP,P).`

Undirected graph represented by its edges, constructing, verify(Vertex)

b

Question 5

Not yet answered

Marked out of 1.00

Flag question

In an undirected graph represented by its edges (e.g. `is_edge(a,b)`), the predicate constructing from below is assumed to build an additional knowledge base of good nodes (`good(Node)`) containing all nodes of the graph for which a certain property holds true. Assuming that `verify(Vertex)` predicate is correctly implemented, and it succeeds if and only if `Vertex` is possessing the underlying property, choose the most specific answer from below:

```
constructing:- is_edge(X,_),
               not(v(X)),
               assertz(v(X)),
               not(verify(X)),
               assertz(good(X)),
               fail.
constructing.
```

Select one:

- ☐ a.
The predicate is correct as it is.
- ☐ b.
The predicate is correct if the predicate `verify` is NOT negated (`not(verify(X))` replaced by `verify(X)`).
- ☐ c. The predicate is correct if the `!` is added after `not(verify(X))`.
- ☐ d.
The predicate is correct if `assertz` is replaced by `asserta`.

non deterministic, non-deterministic, deterministic, first argument is present in the second argument

The predicate below verifies if the first argument (an element) is present in the second argument (a list). To have a non-deterministic call, we need to:

```
member(H, [_|_]).
```

```
member(X, [_|T]) :-
```

```
    member(X, T).
```

Select one:

- ☐ a. add a clause for the empty list
- ☒ b. make the initial call with the first argument free variable
- ☒ c. add a cut in the first clause ✖
- ☐ d. change the order of the clauses

Your answer is incorrect.

concatenates two lists, a nondeterministic call

The predicate below concatenates two lists. To have a nondeterministic call, we need to:

```
append([H|T], L, [H|R]) :-
```

```
    append(T, L, R).
```

```
append([], L, L).
```

Select one:

- ☐ a. change the order of the clauses
- ☐ b. add a clause to decompose the second argument
- ☐ c. make the initial call with all 3 arguments free variables
- ☒ d. make the initial call with the first 2 arguments free variables

Deletes one element from a list, non-deterministically, delete, change

The predicate below deletes one element from a list, non-deterministically. To change it into a deterministic one, we need to:

```
delete(H, [H|T], T).
```

```
delete(X, [H|T], [H|R]) :-
```

```
    delete(X, T, R).
```

Select one:

- ☒ a. add a fact on the empty list and a cut in the first clause
- ☒ b. add a cut in the first clause
- ☐ c. add a fact on the empty list and add a recursive call in the (currently) first clause
- ☐ d. add a recursive call in the first clause

[Clear my choice](#)

replace all occurrences of an element, non-deterministic

The predicate below replaces all occurrences of an element from a list with another element. It has an incorrect non-deterministic behavior, because:

```
replace(_,_,[],[]).
replace(H,A,[H|T],[A|R]):-
    replace(H,A,T,R).
replace(X,A,[H|T],[H|R]):-
    replace(X,A,T,R).
```

Select one:

- ☐ a.
it has a clause for the empty list ✖
- ☒ b. it has no cut in the second clause
- ☐ c. it has no cut in the first clause
- ☐ d.
it has a recursive call in the second clause (it shouldn't)

del_min,del_max, minimum, write, writeln

The code below deletes the minimum element in a list. We executed it on list L, which we know to look as L=[6,]. Given that, according to the specific write/writeln statements, the execution displays what you see below, how did the initial list L look like?

```
del_min([H|T],[H|R],M):-
    del_min(T,R,M),
    M<H,!,
    write(R), write(' '),writeln(M).
del_min([H|T],T,H):-
    write(T), write(' '),writeln(H).
```

?-L=[6,...], del_min(L,New_List,Min).

[]5
[5]4
[4, 5]2
[4, 5]2
[4, 4, 5]2
[3, 4, 2, 4, 5]2
[3, 4, 2, 4, 5]2

[6, 2, 3, 4, 2, 4, 5]

Answer: L = [6, 5, 4, 2, 2, 2]

deletes the maximum element in a list, del_max, write, writeln

The code below deletes the maximum element in a list. We executed it on list L, which we know to look as $L = [7, \dots]$. Given that, according to the specific write/writeln statements, the execution displays what you see below, how did the initial list L look like?

```
del_max([H|T], [H|R], M) :-  
    del_max(T, R, M),  
    M > H, !,  
    write(R), write(' '), writeln(M).  
del_max([H|T], T, H) :-  
    write(T), write(' '), writeln(H).  
  
?-L=[7,...], del_max(L, New_List, Min).
```

```
[]6  
[6]7  
[7,6]8  
[7,6]8  
[5,8,7,6]8  
[5,8,7,6]8  
[4,5,8,7,6]8
```

[7, 4, 8, 5, 8, 7, 6]

Answer: L = [7,

 **Snip saved to clipboard**
Select here to mark up and share the image

delete the minimum element in a list

The code below deletes the minimum element in a list. We executed it on list L, which we know to look as $L = [8, \dots]$. Given that, according to the specific write/writeln statements, the execution displays what you see below, how did the initial list L look like?

```
del_min([H|T], [H|R], M) :-  
    del_min(T, R, M),  
    M < H, !,  
    write(R), write(' '), writeln(M).  
del_min([H|T], T, H) :-  
    write(T), write(' '), writeln(H).
```

```
[]8  
[8]6  
[6,8]1  
[6,8]1  
[3,1,6,8]1  
[3,1,6,8]1  
[4,3,1,6,8]1
```

[8, 4, 1, 3, 1, 6, 8]

append, concatenating 2 and 3 lists, append, append3

The predicates below are concatenating 2 and 3 lists, respectively. If the lists in append3 have lengths l_1 , l_2 and l_3 , respectively, then append3 takes:

```
append([H|T], L, [H|R]) :-
```

```
    append(T, L, R) .
```

```
append([], L, L) .
```

```
append3(A, B, C, R) :-
```

```
    append(B, C, P) ,
```

```
    append(A, P, R) .
```

Select one:

☐ a. $O(l_1 + l_2 + l_3)$

☒ b. $O(l_1 + l_2)$

☐ c. $O(2l_1 + l_2)$

☐ d. $O(l_1 + 2l_2)$

concatenates 2 lists, nondeterministic, append

The predicate below concatenates two lists. To have a nondeterministic call, we need to:

```
append([H|T], L, [H|R]) :-
```

```
    append(T, L, R) .
```

```
append([], L, L) .
```

Select one:

☐ a. change the order of the clauses

☐ b. add a clause to decompose the second argument

☐ c. make the initial call with all 3 arguments free variables

☒ d. make the initial call with the first 2 arguments free variables

Grile rezolvate

The screenshot shows a Moodle quiz interface with three questions. The browser address bar indicates the URL: <https://moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=3816>. The user's name, Daiana Toader, is visible in the top right corner.

Question 1
Partially correct
Mark 0.83 out of 1.00
Flag question

Completati predicatul urmator astfel incat sa gaseasca un drum într-un graf orientat cu lungimea mai mare decât M.

$p(X, Y, M, P) :- p(X, Y, M, [X], P).$
 $p(X, X, M, PP, PP) :-$
length(PP,) ☒ ✓, L).

$p(X, Y, M, PP, FP) :-$
edge(X, Z),
not(member(Z,) ☒ ✓, PP)),
 $p(, Y, M,)$ ☒ ✓, Y, M, ☒ ✓, FP).

Question 2
Correct
Mark 1.00 out of 1.00
Flag question

Alegeți întrebarea care adaugă elementul "a" la sfârșitul unei liste diferență ce conține elementele 1, 2 și 3.

$q(X, SL, [X|EL], SL, EL).$

Select one or more:

- ☐ a. $?-q(a, [1, 2, 3], [], RS, RE).$
- ☐ b. $?-q(a, [1, 2, 3], [], RS, RE).$
- ☐ c. $?-q(a, [1, 2, 3], [], RE).$
- ☒ d. $?-q(a, [1, 2, 3], L, RS, []).$ ✓

Question 3
Correct
Mark 1.00 out of 1.00
Flag question

Se dă predicatul de mai jos:

$q(X, SL, [X|EL], SL, EL).$

Care este răspunsul la următoarea întrebare:

$?-q(3, [1, 2, 3], L, SL, []).$

Select one or more:

- ☐ a. false
- ☐ b. $SL = [1, 2, 3, 3]$
- ☒ c. $L = [3], SL = [1, 2, 3, 3]$ ✓
- ☐ d. $L = [3], SL = [1, 2, 3]$

Activate Windows
Go to Settings to activate Windows.

Vezi ca la aia cu assert nu e correct, e o varianta corecta mai jos

Quiz 3
<https://moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=3815>
 moodle-cs-utcn
 Emilian Popa

Question 1
 Partially correct
 Mark 0.80 out of 1.00
 Flag question

Completați predicatul de mai jos astfel încât să traverseze arborele în **post-ordine** folosind liste diferență.

$p_dl(nl, R, \text{[R]})$ ✓
 $p_dl((K, L, R), \text{[LS]})$ ✓ $(LE) \rightarrow p_dl(L, \text{[ML]})$ ✓ $p_dl(R, \text{[R]})$ ✓ ML
 [KLE])

Question 2
 Correct
 Mark 1.00 out of 1.00
 Flag question

Predicatul de mai jos trebuie să inverseze o listă folosind liste diferență.

$reverse([], _)$
 $reverse([H|T], Rs, Re) \rightarrow reverse(T, Rs, [H|Re]).$
 Ce trebuie modificat?

Select one or more:

- ☐ a. Modifică clauza 2 în: $reverse([H|T], [H|Rs], Re) \rightarrow reverse(T, Rs, Re)$ și clauza 1 în: $reverse([], L, L)$.
- ☐ b. Modifică clauza 2 în: $reverse([H|T], Rs, [H|Re]) \rightarrow reverse(T, Rs, Re)$ și clauza 1 în: $reverse([], L, L)$.
- ☐ c. Schimbă ordinea clauzelor.
- ☒ d. Modifică clauza 1 în: $reverse([], L, L)$ ✓

Question 3
 Incorrect
 Mark 0.00 out of 1.00
 Flag question

Se dă următoare secvență de întrebări:

?- assertz(n(1)), assertz(n(2)), asserta(n(3)).
 ?- retract(n(X)),!, fail.
 Cum va arăta baza de cunoștințe după execuția celor 2 întrebări?

Select one or more:

- ☒ a. Nimic ✗
- ☒ b. n(2) ✓
- ☐ c. n(2), n(3)
- ☐ d. n(2), n(1)

Activate Windows
 Go to Settings to activate Windows.

Question 2- c.

Quiz 3

https://moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=3854

moodle-cs-utcn

LOVE

Programare Logică 2016

VINE VACANTA

Dashboard > Cursul nivel licență / Bachelor of Science Courses > Programare, Limbaje și Algoritmi/ Computer Programming, Languages and Algorithms > 2016/2017 Academic Year > PL 2016 > 19 December - 25 December > Quiz 3

QUIZ NAVIGATION

1 2 3

Show one page at a time

Finish review

Started on Monday, 19 December 2016, 2:12 PM

State Finished

Completed on Monday, 19 December 2016, 2:26 PM

Time taken 14 mins

Marks

Grade

Question 1

Partially correct

Mark 3.00 out of 5.00

Flag question

Completati umătoari predicat astfel încât să găsească ciclurile dintr-un graf care încep cu nodul X.

$\text{cyc}(X, Y, PP, PP1) :- \text{arc}(X, Y), \text{cyc}(Y, PP1)$

$\text{arc}(X, Y, Z, Z1)$

$\text{notmember}(Z, PP1)$

$\text{cyc}(X, Y, PP, PP1) :- \text{arc}(X, Y), \text{cyc}(Y, PP1)$

Question 2

Incorrect

Mark 3.00 out of 5.00

Flag question

Predicatul de mai jos trebuie să găsească primul drum simplu (și doar primul) între 2 noduri din graf (reprezentat cu predicate edge).

$\text{path}(X, Y, Path, Res) :- \text{edge}(X, Y), \text{notmember}(Y, Path), \text{path}(Y, [Y|Path], Res)$

Ce trebuie modificat?

Select one or more:

a. Modifică clauza 1 în: $\text{path}(X, Y, Path, Res) :- \text{edge}(X, Y), \text{notmember}(Y, Path), \text{path}(Y, [Y|Path], Res)$

b. Modifică clauza 2 în: $\text{path}(X, Y, Path, Res) :- \text{edge}(X, Y), \text{notmember}(Y, Path), \text{path}(Y, [Y|Path], Res)$

c. Modifică clauza 2 în: $\text{path}(X, Y, Path, Res) :- \text{edge}(X, Y), \text{notmember}(Y, Path), \text{path}(Y, [Y|Path], Res)$

d. Modifică clauza 1 în: $\text{path}(X, Y, Path, Res) :- \text{edge}(X, Y), \text{notmember}(Y, Path), \text{path}(Y, [Y|Path], Res)$

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

Se dă predicatul de mai jos:

$\text{q}(X, SL, [EL], SL, EL)$

Care este răspunsul la următoarea întrebare:

$?q(3, [1, 2, 3], L, SL, L)$

Select one or more:

a. $L = [3], SL = [1, 2, 3]$

b. false

c. $SL = [1, 2, 3, 3]$

d. $L = [3], SL = [1, 2, 3, 3]$

Activate Windows
Go to Settings to activate Windows.

Course: Logic Program... 7_Deep_Lists.docx Quiz 1

https://moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=2765

moodle-cs-utcn

Andras Hiller

Question 1

Correct

Mark 1.00 out of 1.00

Flag question

The following predicate should remove duplicates from a list. Is it correct? If not, choose the necessary corrections:

$\text{rem}([H|T], R) :- \text{member}(H, T), \text{rem}(T, R)$

$\text{rem}([H|T], R) :- \text{rem}(T, R)$

$\text{rem}([], [])$

$? \text{rem}([1, 2, 3, 2, 4], R)$

$R = [3, 2, 4, 1]$

no.

Select one or more:

a. Not correct, change clause 1 to: $\text{rem}([H|T], [H|R]) :- \text{member}(H, T), \text{rem}(T, R)$

b. Not correct, change clause 2 head to: $\text{rem}([H|T], [H|R]) :-$

c. It is correct.

d. Not correct, change clause 1 head to: $\text{rem}([H|T], [H|R]) :-$

Question 2

Incorrect

Mark 0.00 out of 1.00

Flag question

Given the predicate:

$\text{s}([H|T], L, [H|R]) :- \text{s}(T, L, R)$

$\text{s}([], R, R)$

How many solutions (i.e. corresponding to 'yes' answers) does the following query have:

$? \text{s}([1, 2, 3, 4], B)$

Select one or more:

a. 5

b. 4

c. 0

d. 1

Question 3

Partially correct

Mark 0.87 out of 1.00

Flag question

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). Use H whenever you need a NEW variable for an element from the list.

$\text{one_pass}([H1, H2|T], H1, H2, T, R) :- \text{H1} < \text{H2}, \text{one_pass}(T, H1, H2, R)$

$\text{one_pass}([H1, H2|T], H2, H1, T, R) :- \text{one_pass}(T, H1, H2, R)$

$\text{one_pass}([], H1, H2, T, R) :-$

$\text{one_pass}([], [], R)$

$? \text{one_pass}([3, 1, 4, 5, 2], R)$

$R = [1, 3, 4, 2, 5]$

no.

Activate Windows
Go to Settings to activate Windows.

Quiz 3

SWISH - SWI-Prolog for

←

→

https://moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=3814

Apps

For quick access, place your bookmarks here on the bookmarks bar. Import bookmarks now...

moodle-cs-utcn

Ioana Elena Mocan

Question 1

Partially correct

Mark 0.80 out of 1.00

Flag question

Completați predicatul de mai jos astfel încât să traverseze arborele în **pre-ordine** folosind liste diferență.

$p_dl(nil, T, \text{[]})$ ✓

$p_dl(i(K,L,R), \text{[K|L|R]}, \text{[]})$ ✗ LE $p_dl(L, LS, \text{[]})$ ✓ ML $p_dl(R, \text{[]})$ ✓ ML LE ✓

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Alegeți întrebarea care adaugă elementul "a" la sfârșitul unei liste diferență ce conține elementele 1, 2 și 3.

$q(X, SL, [X|EL], SL, EL)$

Select one or more:

- ☐ a. $?-q(a, [1,2,3], _ , RS, RE)$
- ☐ b. $?-q(a, [1,2,3], [], RS, RE)$
- ☐ c. $?-q(a, [1,2,3], _ , [], RE)$
- ☒ d. $?-q(a, [1,2,3], L, RS, []).$ ✓

Question 3

Incorrect

Mark 0.00 out of 1.00

Flag question

Se dă predicatul de mai jos:

$q(X, SL, [X|EL], SL, EL)$

Care este răspunsul la următoarea întrebare:

$?-q(3, [1,2,3], L, SL, []).$

Select one or more:

- ☒ a. $L = [3], SL = [1,2,3]$ ✗
- ☐ b. $L = [3], SL = [1,2,3,3]$
- ☐ c. $SL = [1,2,3,3]$
- ☐ d. false

Finish review

Windows

Search

Taskbar

System tray

Question 1

Correct

Mark 1.00 out of 1.00

Flag question

Completați următorul predicat astfel încât să găsească ciclurile dintr-un graf care încep cu nodul X.

$c(X, [X \checkmark | P]) :- \text{arc}(X, Y), c(Y, X, [Y], P).$

$c(X, Y, PP, FP) :-$

$\text{arc}(X \checkmark, Z),$

$\text{not}(\text{member}(Z \checkmark, PP)),$

$c(Z, Y \checkmark, [Z|PP], FP).$

$c(X, X \checkmark, PP, P) :- \text{reverse}(PP, P).$

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Predicatul de mai jos trebuie să inverseze o listă folosind liste diferență.

$\text{reverse}([], _, _).$

$\text{reverse}([H|T], [H|Rs], Re) :- \text{reverse}(T, Rs, Re).$

Ce trebuie modificat?

Select one or more:

- ☐ a. Schimbă ordinea clauzelor.
- ☐ b. Modifică clauza 2 în: $\text{reverse}([H|T], Rs, [H|Re]) :- \text{reverse}(T, Rs, Re).$ și clauza 1 în: $\text{reverse}([], L, L).$
- ☒ c. Modifică clauza 2 în: $\text{reverse}([H|T], Rs, Re) :- \text{reverse}(T, Rs, [H|Re]).$ și clauza 1 în: $\text{reverse}([], L, L).$ ✓
- ☐ d. Modifică clauza 1 în: $\text{reverse}([], L, L).$

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

Se dă următoare secvență de întrebări:

?- assertz(n(1)), assertz(n(2)), asserta(n(3)).

?- retract(n(X)),!, fail.

Cum va arăta baza de cunoștințe după execuția celor 2 întrebări?

Select one or more:

- ☒ a. n(1).
n(2). ✓
- ☐ b. n(2).
n(3).
- ☐ c. n(2).
n(1).
- ☐ d. Nimic

Question 2

Incorrect

Mark 0.00 out of 1.00

Flag question

Correct the predicate below such that it can convert both complete and incomplete lists into a diff list:

`convert([], E, E).`

`convert([H|T], S, E):- convert(T, [H|S], E).`

Select one or more:

- ☐ a. change first clause to `convert(L, E, E):- var(L), !.`
and second clause to `convert([H|T], S, E):- convert(T, S, [H|E]).`

- ☒ b. add as first clause `convert(L, E, E):- var(L), !.` ✓

and correct last clause to `convert([H|T], [H|S], E):- convert(T, S, E).`

- ☒ c. add as first clause `convert(L, E, E):- var(L), !.` and correct second clause to `convert([H|T], S, E):- convert(T, S, [H|E]).`

✗

- ☐ d. change first clause to `convert(L, E, E):- var(L), !.`

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

Complete the predicate below to perform the inorder traversal, using difference lists.

`inord_dl(nil, L, [L] ✓).`

`inord_dl(t(K,L,R),LS,LE):-inord_dl(L, [LS] ✓, [K|ML] ✓), inord_dl(R, [R] ✓, ML, [LE] ✓).`

Question 1

Incorrect

Mark 0.00 out of 1.00

Flag question

You are given the predicate below and the example graph on the right (undirected, *edge-clause* form, nodes appear in lexicographic order):

`d(X):- asserta(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).`

What will the predicate base look like after the execution of:

?- d(a).

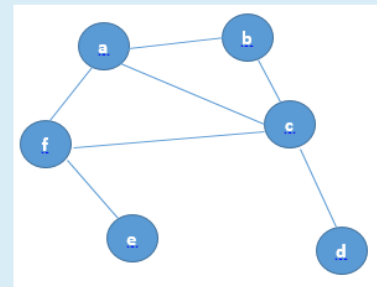
Select one or more:

- ☐ a. `vert(a).`
`vert(b).`
`vert(c).`
`vert(d).`
`vert(f).`
`vert(e).`

- ☒ b. `vert(d).`
`vert(c).`
`vert(b).`
`vert(a).` ✗

- ☒ c. `vert(e).`
`vert(f).`
`vert(d).`
`vert(c).`
`vert(b).`
`vert(a).` ✓

- ☐ d. `vert(a).`
`vert(b).`
`vert(c).`



The predicate below deletes the first occurrence of an element from a list.

$d(X, [X|T], R) :- !$

$d(X, [_|T], [_|R]) :- d(X, T, R).$

$d(, [], []).$

Choose the correct modification such that the resulting predicate deletes all occurrences of the element from the list.

?-d(3, [4,3,2,5,3,2,3], R).

R=[4,2,5,2].

no.

Select one or more:

- ☐ a. Clause 2 should become: $d(X, [], T)$
- ☐ b. Clause 1 should become: $d(X, [X|T], [X|R]) :- !, d(X, T, R)$
- ☒ c. Clause 2 should become: $d(X, [_|T], R) :- d(X, T, R)$ ✗
- ☒ d. Clause 1 should become: $d(X, [X|T], R) :- !, d(X, T, R)$ ✓

- ☐ e. Clause 1 should become: $d(X, [X|T], [X|R]) \rightarrow d(X, T, R)$
- ☒ c. Clause 2 should become: $d(X, [_|T], R) \rightarrow d(X, T, R)$ ✗
- ☐ d. Clause 1 should become: $d(X, [X|T], R) \rightarrow d(X, T, R)$

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

I want to write a predicate which replaces the first (and only the first) occurrence of element *Old* with element *New*, in a list. Help me complete the predicate below to achieve this task:

`replace(Old, New, [Old|T], [New|T]).`

`replace(Old, New, [_|T], [H|R]) :- replace_all(Old, New, T, R).`

`replace(_, _, [], []).`

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

Given the predicate below:

$f([H|T], Acc, R) \rightarrow f(T, [H|Acc], R)$

$f([], R, R)$

What is the answer to the query:

$?- f([1,2,3], [4,5], R)$

Select one or more:

☐ a. yes, $R=[4,5,1,2,3]$

☐ b. yes, $R=[1,2,3,4,5]$

☒ c. yes, $R=[3,2,1,4,5]$ ✓

☐ d. yes, $R=[4,5,3,2,1]$

☐ e. no

$?- insert_ord([1,2,3,6], R)$

$R=[1,2,3,3,6]$

no.

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Given the predicate:

$s([H|T], L, [H|R]) \rightarrow s(T, L, R)$

$s([], R, R)$

How many solutions (i.e. corresponding to 'yes' answers) does the following query have:

$?- s([], B, [1,2,3,4])$

Select one or more:

☒ a. 5 ✓

☐ b. 0

☐ c. 4

☐ d. 1

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

The following predicate performs the union of two sets (represented as lists, member is considered known and implemented):

$u([H|T], L, R) \rightarrow \text{member}(H, L), !u(T, L, R)$

$u([H|T], L, [H|R]) \rightarrow u(T, L, R)$

$u([], L, L)$

Select from the alternatives below the one which correctly modifies the predicate such that it performs the intersection of two sets, instead of the union.

Select one or more:

☐ a. Change clause 1 head to: $u([H|T], L, [H|R]) \rightarrow$

☐ b. Change clause 3 to: $u([], L, [])$

☒ c. Change clause 1 head to: $u([H|T], L, [H|R]) \rightarrow$, the head of clause 2 to: $u([H|T], L, R)$ and change clause 3 to: $u([], L, [])$ ✓

☐ d. Change clause 1 body to: $\rightarrow \text{member}(H, L), !u(T, L, R)$

Activate Windows
Go to Settings to activate Windows.

6

Courses » Programare, Limbaje și Algoritmi/ Computer Programming, Languages and Algorithms » 2016/2017 Academic Year » LP2016 » 7 November - 13 November » Quiz 1

Started on Tuesday, 8 November 2016, 10:10 AM

State Finished

Completed on Tuesday, 8 November 2016, 10:22 AM

Time taken 12 mins

Marks 2.67/3.00

Grade 8.89 out of 10.00 (89%)

Question 1

Partially correct

Mark 0.67 out of 1.00

Flag question

The following predicate should insert an element in an ordered list. Fill in the gaps to complete its specification (see the example query before you submit your answer !!)

insert_ord(X, [H|T], [HIRI] X) - X >= H, !, insert_ord(X, T, R)

insert_ord(X, T, [X|T])

?-insert_ord(3, [1,2,3,6],R).

R=[1,2,3,3,6];

no.

Question 2

Correct

Given the predicate:

s([H|T], L, [HIRI]) :- s(T, L, R)

Question 1

Correct

Mark 1.00 out of 1.00

Flag question

Given the predicate below:

 $f([H|T], \text{Acc}, R) \text{ :- } f(T, [H|\text{Acc}], R).$ $f([], R, R).$

What is the answer to the query:

 $? \text{ :- } f([1,2,3],[4,5],R).$

Select one or more:

- ☐ a. yes, $R=[1,2,3,4,5]$
- ☒ b. yes, $R=[3,2,1,4,5]$ ✓
- ☐ c. yes, $R=[4,5,1,2,3]$
- ☐ d. yes, $R=[4,5,3,2,1]$
- ☐ e. no

Question 2

Incorrect

Mark 0.00 out of 1.00

Flag question

The following predicate performs the intersection of two sets (represented as lists, member is considered known and implemented):

 $i([H|T], L, [H|R]) \text{ :- } \text{member}(H, L), !, i(T, L, R).$ $i([H|T], L, R) \text{ :- } i(T, L, R).$ $i([], _, []).$

Select from the alternatives below the one which correctly modifies the predicate such that it performs the difference of two sets, instead of the intersection.

Select one or more:

- ☐ a. Change clause 3 to: $i([], L, L).$
- ☒ b. Change clause 1 body to: $\text{not member}(H, L), !, i(T, L, R).$ ✓
- ☒ c. Change clause 1 head to: $i([H|T], L, R) \text{ :- } \dots$ ✗
- ☐ d. Change clause 1 head to: $i([H|T], L, R) \text{ :- } \dots$ and clause 3 to: $i([], L, L).$

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

I want to write a predicate which replaces all the occurrences of element *Old* with element *New*, in a list. Help me complete the predicate below to achieve this task: $\text{replace_all}(\text{Old}, \text{New}, [\text{Old}|T], [\text{New}|R]) \text{ :- } \text{replace_all}(\text{Old}, \text{New}, T, R).$ $\text{replace_all}(\text{Old}, \text{New}, [H|T], [H|R]) \text{ :- } \text{replace_all}(\text{Old}, \text{New}, T, R).$ $\text{replace_all}(_, _, [], []).$

Quiz 2 - Deep lists, trees

Facebook

https://moodle.cs.utdallas.edu/mod/quiz/review.php?attempt=3484

moodle-cs-utdallas

Markus Bolognani

e. 3

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

I want to write a predicate which searches for an element in an incomplete list. Choose the correct option to complete the code below.

search_ll(X, [_|_]) :- search_ll(X, T).

Select one or more:

☐ a. Add as first two clauses:
search_ll(X, L) :- var(L), !.

☐ b. Add as first clause: search_ll(X, L) :- var(L), !, fail.

☐ c. Add as first clause: search_ll(X, [_|_]) :- !.

☒ d. Add as first two clauses:
search_ll(X, L) :- var(L), !, fail.
search_ll(X, [_|_]).

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

The following predicate searches for a certain key in an incomplete BST:

search_ll(_ T) :- var(T), !, fail.

search_ll(Key, t(K, L, R)) :- !, fail.

search_ll(Key, t(K, L, R)) :- Key < K, !, search_ll(Key, L).

search_ll(Key, t(K, L, R)) :- search_ll(Key, R).

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

Select one or more:

☐ a. Change the first clause to: search_ll(_ T) :- var(T), !.

☐ b. Remove the first clause and change the second clause to: search_ll(Key, T) :- var(T), T = t(Key, _).

☒ c. Remove the first clause.

☐ d. Change the first clause to: search_ll(Key, T) :- var(T), !, T = t(Key, _).

Activate Windows
Go to Settings to activate Windows.

10:26 AM
12/6/2016

https://moodle.cs.utcluj.ro/mod/quiz/review.php?attempt=3422

Pinxos Projects Proje

moodle-cs-utcn

Roxana-Adriana Cimpan

Flag question

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Următorul predicat verifică dacă un element există într-o listă completă.

$m(X[_{[T]}]) \rightarrow t$

$m(X[_{[T]}]) \rightarrow m(X.T)$

Modificati predicatul astfel încât să verifice dacă un element există într-o listă incompletă.

Select one or more:

☒ a. Adaugă ca primă clauză: $m(_{[L]}) \rightarrow \text{var}(L), t$, fail ✓

☐ b. Adaugă ca primă clauză: $m(_{[L]}) \rightarrow \text{var}(L), t$

☐ c. Adaugă ca primă clauză: $m(_{[L]}) \rightarrow \text{var}(L), \text{fail}$

☐ d. Adaugă ca primă clauză: $m(_{[L]}) \rightarrow \text{var}(L), t$, fail și șterge "t" din clauza doi.

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

Aveni următorul predicat:

$s(_{[L]}) \rightarrow \text{var}(L), \text{fail}$

$s(X[_{[L]}])$

$s(X[_{[T]}]) \rightarrow s(X.T)$

Câte răspunsuri de yes/true primesc la întrebarea

$? X=3, s(X[3,3,3])$

Select one or more:

☐ a. 4

☐ b. 1

☐ c. 0

☒ d. Infinit ✓

☐ e. 3

Activate Windows
Go to Settings to activate Windows.

You are logged in as Roxana-Adriana Cimpan (Log out)

PL2016

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Complete the following predicate to compute the height of an incomplete binary tree:

```
height(T, 0):- var(T) ✓, [ ] ✓.
```

```
height(t(____,L,R), H):- height(L [ ] ✓, HL), height(R [ ] ✓, HR), max([HL [ ] ✓, HR [ ] ✓], HH),
H [ ] ✓ is HH [ ] ✓ + 1.
```

```
?- height(t(4,t(2,____),____), R).
```

```
R = 2 ;
```

```
false
```

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

The following predicate searches for a certain key in a complete BST.

```
search_it(Key, t(Key, __, __)):- !.
```

```
search_it(Key, t(K, L, R)):- Key < K, !, search_it(Key, L).
```

```
search_it(Key, t(____, __, R)):- search_it(Key, R).
```

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

Select one or more:

☐ a. Change the first clause to: `search_it(Key, T):-var(Key),!,T=t(Key,____).`
☒ b. Do nothing. ✓

☐ c. Change the first clause to: `search_it(____, T):-var(T), !.`
☐ d. Add, as first clause: `search_it(Key, T):-var(T), !, T=t(Key,____),fail.`

https://moodle.cs.utcluj.ro/moodle/quiz/review.php?attempt=3422

Reitor Proiectu Propriu

moodle-cs-utcn

Roxana-Adriana Cîrșan

Programare Logică 2016

Dashboard > Cursuri nivel licență / Bachelor of Science Courses > Programare, Limbaje și Algoritmi/ Computer Programming, Languages and Algorithms > 2016/2017 Academic Year > PL 2016 > 5 December - 11 December > Quiz 2

QUIZ NAVIGATION

1

2

3

✓

✓

✓

Show one page at a time

Finish review

Started on Monday, 5 December 2016, 10:02 AM

State Finished

Completed on Monday, 5 December 2016, 10:13 AM

Time taken 11 mins 48 secs

Marks 3.00/3.00

Grade 10.00 out of 10.00 (100%)

Question 1

Correct

Mark 1.00 out of 1.00

Flag question

Completați predicatul astfel încât să insereze un element la sfârșitul listei incomplete.

insert_at(X, l) :- l.

insert_at(X, l) :- insert_at(X, T).

Question 2

Correct

Mark 1.00 out of 1.00

Flag question

Următorul predicat verifică dacă un element există într-o listă completă

m(X,[X]):-1.

m(X,[_],_):-m(X,T).

Modificați predicatul astfel încât să verifice dacă un element există într-o listă incompletă.

Select one or more:

- ☒ a. Adaugă ca primă clauză: m(_L):-var(L), !, fail.
- ☐ b. Adaugă ca primă clauză: m(_L):-var(L), !.
- ☐ c. Adaugă ca primă clauză: m(_L):-var(L), fail.
- ☐ d. Adaugă ca primă clauză: m(_L):-var(L), !, fail, și șterge "!" din clauza doi.

Question 3

Correct

Mark 1.00 out of 1.00

Flag question

Avem următorul predicat:

s(_L):-var(L), fail.

s(X,[X]):-1.

s(X,[_],_):-s(X,T).

Câte răspunsuri de adevărat primesc la întrebarea

Activate Windows

Go to Settings to activate Windows.

10:22 AM

12/5/2016

[illegible]

Question 1

Partially correct

Mark 0.89 out of

1.00

Flag question

Completați predicatul astfel încât să aplatizeze o listă adâncă *incompletă*.flatten_il(L, , ☒ var(L) ✓, I).flatten_il([H|T], [H|R] ✓, ☒ atomic(H), , flatten_il(T, , R).flatten_il([H|T], ☒ R, ☒ flatten_il(H, , ☒ flatten_il(T, , ☒ append(R1, R2, , ☒).

Ex apel:

?- flatten_il([1, [2|_] | _], R).

R = [1,2];

no

Question 2

Correct

Mark 1.00 out of

1.00

Flag question

Următorul predicat verifică dacă un element există într-o listă completă:

 $m(X, [X|_]) :- !.$ $m(X, [_|T]) :- m(X, T).$

Modificați predicatul astfel încât să verifice dacă un element există într-o listă adâncă și să fie nedeterminist.

Select one or more:

- ☒ a. Adaugă clauza: $m(X, [H|_]) :- m(X, H).$ și șterge "!" din prima clauză ✓
- ☐ b. Schimba a doua clauză în: $m(X, [H|T]) :- m(X, H), !, m(X, T).$
- ☐ c. Schimba a doua clauză în: $m(X, [H|T]) :- m(X, H), m(X, T).$ și șterge "!" din prima clauză
- ☐ d. Adaugă între prima și a doua clauză: $m(X, [H|_]) :- m(X, H).$

Question 3

Correct

Mark 1.00 out of

1.00

Flag question

Avem următorul predicat:

 $s([], L) :- var(L), !, fail.$ $s(X, [X|_]) :- !.$ $s(X, [_|T]) :- s(X, T).$

Câte răspunsuri de yes/true primesc la întrebarea:

?- X=3, s(X,[3,3,3|_]).

Select one or more:

- ☒ a. 1 ✓
- ☐ b. Infinit
- ☐ c. 4
- ☐ d. 0

